

OGP: A BGP-Inspired Federation Protocol for Human-Mediated AI Gateway Interoperability

David Proctor
Trilogy AI Center of Excellence
USA
david.ai@trilogy.com

ABSTRACT

AI assistants are proliferating but remain organizationally siloed. When users from different organizations need their agents to collaborate—on legal review, insurance authorization, or shared project coordination—no protocol exists for establishing the required cross-boundary trust. Existing protocols address adjacent problems: MCP attaches tools to single agents, A2A delegates tasks within organizational boundaries, and ANP discovers agents on the open web. None address the fundamental challenge of two humans granting their AI systems permission to work together directly, under explicit scoped authorization, with unilateral revocability.

We present the Open Gateway Protocol (OGP), a federation protocol that borrows its trust and policy model from BGP (Border Gateway Protocol). OGP treats each AI gateway as an autonomous system and establishes bilateral peer relationships through human-approved handshakes, per-peer scope negotiation, and Ed25519 per-message authentication. A three-layer Doorman enforces scope at runtime. An OSPF-inspired state machine tracks federation health over time. We describe the protocol design, analyze its security properties against a published threat taxonomy, and present microbenchmark results showing that the cryptographic and enforcement overhead is sub-millisecond at every layer, adding negligible latency to agent-to-agent communication. OGP is implemented as an open-source reference daemon (`@dp-pcs/ogp`, v0.11.3) with 386 commits over three months and active deployments.

1 INTRODUCTION

The past year has produced a proliferation of AI agent protocols. MCP [2] standardizes how agents attach to tools. A2A [4] standardizes how agents delegate tasks to one another. ANP [1] proposes a decentralized open web for agent discovery. Yet none of these protocols address a problem that arises the moment two people try to use their AI assistants together: *how do two gateways owned by different humans establish trust?*

Consider two concrete scenarios:

Legal review. Alice’s firm is reviewing a contract with Bob’s firm. Alice’s AI assistant has context on her firm’s position; Bob’s has context on his. Neither party wants to share raw context with the other—they want their agents to exchange structured, scoped, attributable information without either firm’s data crossing an uncontrolled boundary.

Project coordination. David’s agent and Cosmo’s agent are collaborating on a cross-organizational software project. They need to exchange status updates, query each other’s project decisions, and push signed contributions to a shared record—but David does not want Cosmo’s agent to access his calendar, his email context, or any capability he has not explicitly granted.

No existing protocol addresses these scenarios. MCP runs within a single trust boundary. A2A assumes a shared organizational identity provider and targets intra-organizational orchestration [3]. ANP provides identity and discovery for the open public web but has no bilateral approval mechanism—any authenticated agent can connect to any other [1]. ActivityPub [7] federates social content, not agentic tasks.

We present **Open Gateway Protocol (OGP)**, which fills this gap by treating AI gateway federation the same way BGP treats autonomous network federation: bilateral human approval, explicit per-peer policy, and cryptographic authentication on every message.

OGP makes four contributions:

- (1) A **gateway-level federation protocol** with a human-mandatory approval handshake, cleanly separating the trust decision (human-made, once) from message exchange (agent-automated, many times).
- (2) A **three-layer scope model** (capability advertisement, per-peer negotiation, runtime enforcement) that separates what a gateway supports from what a specific peer is granted.
- (3) An **OSPF-inspired federation lifecycle state machine** that tracks peer health and enables graceful degradation and tombstoning.
- (4) A **reference implementation** (open source, 386 commits, npm package `@dp-pcs/ogp`) with microbenchmarks demonstrating sub-millisecond overhead at every enforcement layer.

2 PROBLEM STATEMENT

2.1 The Cross-Boundary Trust Gap

Existing agent protocols can be placed on two axes: the trust model they assume (service-level vs. human-level), and the scope they control (single-agent vs. cross-org). Table 1 shows the landscape.

The gap is specific: no protocol makes human approval of federation *mandatory at the protocol level*, enforces *per-peer scoped permissions* on every message, and provides *unilateral*

Table 1: Positioning of AI agent protocols by trust model and scope.

Protocol	Trust model	Scope	Human approval
MCP	Host-delegated	Tool/agent	Guideline only
A2A	OAuth + cards	Intra-org	Not required
ANP	DID crypto	Open web	High-risk only
ACP	Zero-Trust/DID	Multi-org	Future work
ActivityPub	Impl.-delegated	Social	Optional
OGP	Bilateral	Cross-org	Mandatory

revocability with cryptographic tombstoning to prevent silent re-trust.

2.2 Why Existing Approaches Fail

Shared identity providers (OAuth, enterprise SSO) require both parties to be enrolled in the same identity system—which is precisely what fails in cross-organizational scenarios between parties with no pre-existing relationship.

API key exchange provides authentication but no scope model: the API key either works or it does not. There is no per-intent granularity, no topic restrictions, no sliding-window rate limiting, and no revocation semantics beyond deleting the key out-of-band.

Trust-on-first-use (ActivityPub Follow, ANP connection) makes the approval optional or automated. This is appropriate for social networks; it is inappropriate for sensitive cross-organizational AI collaboration where the human must remain the arbiter of what their agent is authorized to do on their behalf.

3 RELATED WORK

MCP [2] (Anthropic, 2024) is a JSON-RPC protocol for attaching tools and resources to a single LLM agent. Trust is entirely delegated to the host application; the spec explicitly states “MCP itself cannot enforce these security principles at the protocol level.” MCP has no message signing, no peer-to-peer federation concept, and no revocation mechanism.

A2A [4] (Google, 2025–2026) enables task delegation between agents via Agent Cards published at `/.well-known/agent.json`. Identity is Agent-Card + OAuth. A2A does not mandate human approval for new agent relationships and targets intra-organizational enterprise orchestration. Its scope control relies on OAuth scopes; a recent security analysis identified “insufficiently granular token scope” as a known risk [5]. A2A and OGP are complementary: OGP establishes cross-org gateway trust; A2A can be used internally to delegate tasks within a trusted boundary.

ANP [1] (2024, open-source) proposes W3C DIDs (`did:wba`) for decentralized agent identity and JSON-LD for semantic capability discovery. ANP is designed for the open public web and prioritizes discovery over bilateral approval. It explicitly flags revocation infrastructure (CRLs, status protocols) as an open problem. The trust model assumes “least trust” but has no mandatory human approval gate for federation.

ACP [8] (IBM/BeeAI, now Linux Foundation) introduces a Zero-Trust Agentic Security model with DID-based identity and per-request Proof-of-Intent (PoI) signatures. ACP’s design is closest to OGP’s in security posture, but it targets infrastructure-scale messaging and makes human approval of federation a roadmap item rather than a protocol invariant. **ActivityPub** [7] (W3C, 2018) federates social content between independently operated servers. Its trust model is implementation-delegated; the spec notes “there are no Ed25519 agreed upon mechanisms for authentication” at standardization time. Message signing is advisory, not mandatory.

BGP [6] provides the conceptual model for OGP’s peering architecture. The peering handshake, bilateral policy negotiation, and peer lifecycle state machine in OGP are all derived from BGP’s design. OGP does not implement route computation, path selection, or multi-hop forwarding—it takes only the peering model.

A recent security threat taxonomy [5] analyzed MCP, A2A, Agora, and ANP and identified 16 risks across three categories. OGP’s design structurally addresses 7 of these (Table 2).

Table 2: Security risks from [5] and OGP’s structural mitigations.

Risk	OGP mitigation
Replay attacks	Per-message nonce + 5-min timestamp window
Identity forgery	Ed25519 verify on every message
Sybil attacks	Human approval gate per peer
Token scope escalation	Runtime Doorman re-evaluates stored grant
Onboarding exploitation	No automated path to federation
Cross-vendor trust exploitation	Gateway is the trust boundary
Intent deception	Doorman rejects intents not in peer’s grant

4 PROTOCOL DESIGN

4.1 Identity and Discovery

Each OGP gateway generates an Ed25519 keypair on first start. The *public key is the identity*: there is no separate identifier that must be resolved against an external registry. On macOS, the private key is stored in an instance-specific Keychain entry; on other platforms it is encrypted at rest using an operator-provided secret.

Gateways publish a signed discovery card at `GET /.well-known/ogp:`

Listing 1: Discovery card (abbreviated).

```
{
  "publicKey": "302a300506032b65...",
  "displayName": "David Proctor / Junior",
  "gatewayUrl": "https://ogp.david.example.com",
  "capabilities": {
    "intents": ["message", "agent-comms",
               "project.contribute", "project.query"],
    "features": ["relay", "rendezvous"]
  },
  "signature": "a1b2c3..."
}
```

An optional Rendezvous service (`rendezvous.elelem.expert`) provides pubkey lookup and short-lived invite codes (10-minute TTL) without relaying any message content.

4.2 Federation Lifecycle

The handshake is a two-message signed exchange:

- (1) **Request:** Alice sends a signed federation request to Bob's `POST /federation/request`. The payload includes Alice's public key, gateway URL, offered intents, and a freshness timestamp. Bob records the request as `init`-state.
- (2) **Approval:** A human at Bob's gateway approves the request. Bob sends a signed approval to Alice's `POST /federation/approve`. Both gateways transition to `twoWay` state. The first successful bidirectional health check advances to `established`.

Either party can remove the other unilaterally: `ogp federation remove <peer>` sends a signed notification to `POST /federation/removed` and immediately transitions the local record to `tombstoned`. The notification is best-effort; removal is effective regardless of whether it is received. A re-federation attempt from a tombstoned peer creates a fresh `init`-state record requiring explicit re-approval—there is no silent path back to trust.

The full lifecycle follows an OSPF-inspired state machine:

```
init → twoWay → established ↔ degraded ↔ down →
                                tombstoned
```

4.3 Three-Layer Scope Model

OGP enforces scope at three distinct layers, preventing any single bypass from exposing the full capability surface:

Layer 1 — Capability Advertisement The gateway publishes what it *can* support in its discovery card. Peers cannot request capabilities not in this list.

Layer 2 — Per-Peer Grant At approval time, the approving human sets exactly what this specific peer *will* be granted: a set of intents, optional topic restrictions for `agent-comms`, and a per-intent sliding-window rate limit (e.g., 100 requests/3600 s). Grants can be updated at any time without re-federation.

Layer 3 — Runtime Enforcement (Doorman) The Doorman validates every incoming message against the peer's current grant. It checks peer identity, approval status, intent coverage, topic allowlist, and rate limit in sequence before any message reaches the local agent.

The separation between Layer 2 (stored grant) and Layer 3 (runtime evaluation) means grants can be tightened mid-relationship without protocol overhead: the Doorman re-reads the stored grant on each request.

4.4 Agent-to-Agent Communication (Agent-Comms)

Beyond raw message passing, OGP includes a structured `agent-comms` intent for natural-language agent-to-agent dialogue. Messages carry a *topic* (e.g., `project-updates`, `signal`) that is independently scoped in the peer's grant.

The receiving gateway applies a 7-layer policy precedence chain to determine how the local agent should handle the message—from a global default down to a peer-and-topic-specific override:

- (1) Legacy inbound policy mode (safety fallback)
- (2) Global default response level
- (3) Peer-specific default
- (4) Global message-class rule (`human-relay`, `agent-work`, etc.)
- (5) Peer message-class override
- (6) Global topic-level rule
- (7) Peer topic-level override

Response levels range from `full` (share everything) through `summary`, `escalate` (ask the human first), and `off` (signed rejection with reason—not a silent drop).

4.5 Signed Project Contributions

For cross-organizational project coordination, OGP provides a signed contribution system. When a gateway logs a contribution, the CLI:

- (1) Mints a sortable ULID as the stable cross-federation ID
- (2) Constructs a canonical record: `{id, projectId, authorId, entryType, summary, timestamp}`
- (3) Signs the canonical JSON with the gateway's Ed25519 private key
- (4) Stores the signed envelope locally and pushes it to peer gateways

`authorId` is the gateway's Ed25519 public key—no separate key distribution is needed. On receipt, the Doorman verifies: (1) the signature is valid, (2) the signed `authorId` matches the federation-authenticated sender, and (3) the signed `projectId` matches the target project. Any mismatch is a 401. A contribution carries one stable ULID across every gateway it is pushed to.

4.6 Transport Independence

OGP separates the protocol from the transport. The current reference implementation supports three modes:

Direct Peers reach each other via their configured `gatewayUrl` (permanent Cloudflare tunnel, ngrok, VPS, or cloud endpoint).

Relay Messages route through the optional Rendezvous relay for gateways that cannot receive inbound connections directly.

Future The protocol carries a signed transport descriptor in the discovery card, enabling future transports (e.g., Iroh P2P) without protocol changes.

5 SECURITY ANALYSIS

5.1 Threat Model

OGP’s threat model: two gateways, owned by different humans who do not necessarily trust each other’s organizations, each running an AI agent framework behind a TLS endpoint. The adversary can intercept, replay, or forge messages; can attempt to establish unauthorized federation; or can attempt to exceed granted scope.

5.2 Cryptographic Guarantees

All messages carry an Ed25519 signature over the canonical JSON serialization of the payload, including a freshness timestamp. Verification at the Doorman provides:

- **Authenticity:** the message was produced by the holder of the sender’s private key.
- **Integrity:** the payload has not been modified in transit.
- **Replay resistance:** the 5-minute timestamp window and per-message nonces prevent replayed messages from being accepted.
- **Non-repudiation:** each contribution carries its author’s public key in the signed bytes, making forgery detectable.

Key generation uses Node.js’s `node:crypto` Ed25519 implementation. On macOS, private keys are stored in an instance-specific Keychain entry derived from the state directory path; on non-macOS, private keys are encrypted at rest using an operator-provided secret.

5.3 Authorization Guarantees

The three-layer scope model means that a compromised peer cannot exceed its granted permissions even if it can generate valid signatures: the Doorman enforces the grant independently of the message content. A peer granted `agent-comms` on topics `general` and `project-updates` cannot invoke `project.contribute` or send an `agent-comms` message on topic `finance`—both are rejected at Layer 3 before the message reaches the local agent.

5.4 Limitations

OGP does not address: (1) compromise of the gateway host itself (key exfiltration, process injection)—this is outside the protocol’s scope; (2) prompt injection in message content that manipulates the receiving agent’s behavior after delivery—this requires defenses at the agent layer; (3) denial-of-service against the Rendezvous service (which is optional; its failure is graceful).

6 IMPLEMENTATION AND EVALUATION

6.1 Reference Implementation

OGP is implemented as a Node.js daemon (`@dp-pcs/ogp`, MIT license, v0.11.3 as of this writing). The daemon runs

alongside any AI framework on a separate port and communicates with the local agent via `POST /hooks/agent`. The implementation comprises approximately 8,000 lines of TypeScript across 386 commits spanning three months of active development.

Key implementation components:

- `src/shared/signing.ts` Ed25519 sign/verify using Node.js built-in `node:crypto.signCanonical()` adds a freshness timestamp and returns the exact JSON bytes alongside the signature so verifiers operate on the same bytes that were signed.
- `src/daemon/doorman.ts` Three-layer runtime enforcement. Checks peer status, finds the applicable scope grant via `findScopeGrant()`, evaluates topic coverage, and enforces a sliding-window rate limit using an in-memory timestamp store.
- `src/daemon/peers.ts` Peer storage with the full federation lifecycle state machine (`deriveFederationState()`).
- `src/daemon/contribution-signing.ts` Per-contribution ULID generation, canonical serialization, and Ed25519 signing.

The implementation includes a reference Rendezvous server (`packages/rendezvous`), a macOS companion desktop app (`ogp-companion`, Tauri/React), a macOS menu bar monitor, a marketing website (`ogp.elelem.expert`), and four Claude Code skills (`ogp`, `ogp-agent-comms`, `ogp-project`, `ogp-expose`).

6.2 Microbenchmarks

We measured the overhead of OGP’s cryptographic and enforcement layers on Apple M4 Pro (Node.js v26.3.0). All measurements use `performance.now()` with batch timing to achieve sub-microsecond resolution.

Table 3: OGP microbenchmark results (Apple M4 Pro, Node.js v26.3.0, $n \geq 5,000$).

Operation	Mean	p99	Notes
Ed25519 sign	19 μ s	31 μ s	52,632 ops/sec
Ed25519 verify	44 μ s	69 μ s	22,573 ops/sec
HMAC-SHA256 sign	8 μ s	—	2.3 $\times$ faster (weaker identity)
Full msg lifecycle	65 μ s	95 μ s	sign + verify + timestamp
Signed contribution	86 μ s	135 μ s	ULID + sign + verify
3-layer scope eval	<1 μ s	1.2 μ s	capability+grant+rate-limit
7-layer policy chain	<1 μ s	<1 μ s	all precedence layers
Doorman (1 peer)	0.1 μ s	0.7 μ s	Map lookup + grant find
Doorman (1000 peers)	<1 μ s	<1 μ s	hash map; no linear scan

The Ed25519 sign/verify cycle (19 μ s / 44 μ s) dominates the per-message overhead, as expected for an asymmetric signature scheme. The 2.3 $\times$ overhead versus HMAC-SHA256 is the cost of non-repudiable per-message authorship: HMAC provides only shared-secret integrity, not individual identity. The Doorman and policy-resolution layers add sub-microsecond overhead independent of federation scale—the Map-based peer store does not degrade with peer count.

In aggregate, an OGP gateway adds approximately $65\ \mu\text{s}$ per message for the sign+verify+authorization path. For agent-to-agent communication, where message round-trips are measured in seconds (agent reasoning time), this overhead is negligible.

6.3 Deployment Case Study

OGP is deployed between two active gateways: David Proctor’s gateway (at Trilogy, running OpenClaw) and Cosmo’s gateway (a separate organization, also running OpenClaw). The federation was established with explicit mutual human approval on both sides.

The deployed federation uses:

- **agent-comms** scope with topics **general** and **signal**
- **project.contribute** and **project.query** scopes
- Rate limit: 100 requests per hour per intent
- Inbound policy: **summarize** mode (agent summarizes inbound messages before surfacing to the human)

Cosmo’s agent can send natural-language messages to David’s agent on approved topics, push signed project contributions, and query the shared project state. It cannot access David’s calendar, memory, email context, or any capability not in the explicitly granted scope—the Doorman enforces this on every message without the local agent being involved in the access decision.

The bidirectional federation has been maintained across multiple gateway restarts, tunnel URL rotations, and software upgrades without requiring re-federation—the public key is the durable identity, not the gateway URL.

7 DISCUSSION

7.1 What OGP Does Not Solve

OGP solves trust establishment and scope enforcement at the gateway boundary. It does not solve:

NAT traversal OGP requires each gateway to be publicly reachable. The relay transport provides a workaround for gateways behind strict NAT, but peer-to-peer hole punching is out of scope for this protocol version.

Message content semantics OGP delivers authenticated, scoped messages. What the receiving agent does with them—and whether it can be manipulated by prompt injection within the message content—is a matter for the agent layer, not the gateway layer.

Group federation OGP is strictly point-to-point. Multi-party federation (three or more gateways in a mesh) requires pairwise peering; OGP does not define a group membership protocol.

Revocation propagation When a gateway is compromised, its peers must be individually notified. There is no broadcast revocation mechanism.

7.2 OGP Apps: A Distribution Layer

A natural extension of trusted gateway relationships is software distribution through them. OGP Apps (`ogp-app.json`

manifests) let peers advertise declarative capability bundles—specifying which intents they fire, which skills they install into the recipient’s AI agent, and where their output lives. Installation goes through a consent gate (the human sees exactly which scripts run and which intents are used before confirming). The federation trust relationship doubles as a software distribution channel, scoped to the same bilateral approval model.

7.3 Relationship to Emerging Standards

OGP is not a competing standard—it is a complement to the emerging agent protocol ecosystem. We anticipate that OGP gateways will use A2A for intra-org task delegation, MCP for tool attachment, and ANP for public web discovery, while OGP governs what crosses the organizational boundary. The gateway is the right abstraction for this role: a durable, human-controlled trust boundary that outlasts any individual agent session.

The cleanest framing: A2A is HTTP, enabling request/response between services. OGP is BGP, establishing the policy framework between autonomous systems before any requests flow.

8 CONCLUSION

We presented OGP, a federation protocol that applies BGP’s peering model to AI gateway interoperability. OGP’s defining properties—mandatory human approval for federation establishment, three-layer scope enforcement, Ed25519 per-message authentication, and unilateral revocability with cryptographic tombstoning—address a gap that no existing agent protocol fills. The reference implementation demonstrates sub-millisecond enforcement overhead at every layer, confirming that the security model does not impose meaningful latency costs on agent communication. Active deployment between two organizational gateways validates the protocol under real cross-boundary collaboration conditions.

The protocol specification, reference implementation, and benchmarks are available at <https://github.com/dp-pcs/ogp>.

REFERENCES

- [1] ANP Open Source Community. 2024. Agent Network Protocol (ANP) Technical White Paper. arXiv:2508.00007 [cs.NI]
- [2] Anthropic. 2024. *Model Context Protocol*. Technical Report. Anthropic. <https://modelcontextprotocol.io/>.
- [3] Abhinav Bandi et al. 2025. A Survey of Agent Interoperability Protocols: MCP, ACP, A2A, and ANP. arXiv:2505.02279 [cs.MA]
- [4] Google. 2025. *Agent2Agent (A2A) Protocol*. Technical Report. Google. <https://google.github.io/A2A/>.
- [5] Jiahao Hua et al. 2026. Security Threat Modeling for Emerging AI-Agent Protocols: A Comparative Analysis of MCP, A2A, Agora, and ANP. arXiv:2602.11327 [cs.CR]
- [6] Yakov Rekhter, Tony Li, and Susan Hares. 2006. A Border Gateway Protocol 4 (BGP-4). RFC 4271.
- [7] Christopher Webber, Jessica Tallon, Erin Shepherd, Amy Guy, and Evan Prodromou. 2018. *ActivityPub*. W3C Recommendation. W3C. <https://www.w3.org/TR/activitypub/>.
- [8] Cheng Yao, Yiping Chen, et al. 2026. Agent Communication Protocol (ACP) for Federated Orchestration. arXiv:2602.15055 [cs.MA]